

budge

Product Requirements Document — for implementation by Claude Cowork

June 2026 · v1.1 · Owner: Tyler

1. Overview

Build the glue scripts, repository scaffolding, and automation around a plain-text accounting stack composed of **hledger** (system of record), **SimpleFIN Bridge** (bank data), **Paisa** (dashboard), and **git** (history/sync). The deliverable is a small, well-documented toolset that: imports bank transactions daily, categorizes them via deterministic rules first and an AI model second, holds AI-categorized transactions in a reviewable pending state that is still fully visible to the budget, provides a fast vendor-grouped weekly review, and keeps everything versioned and verifiable.

Prime directive — no frankensteining. hledger, Paisa, and SimpleFIN Bridge are used completely stock, configured only through their official mechanisms. Custom code lives only at the seams, communicates only via stable interfaces (SimpleFIN API, CSV, hledger journal format, hledger CLI output), and must be individually replaceable without any change to the data. Do not patch, fork, or wrap the internals of any stock component. **Corollary:** the budge CLI exists only to orchestrate workflows hledger does not have (fetch, categorize, review/promote, plan). It must never wrap, alias, or proxy native hledger functionality — no ``budge balance``, no ``budge report``. Reporting, queries, and validation are operator-facing hledger commands; budge scripts may shell out to the hledger binary internally, but the accounting interface the operator learns and uses is hledger itself.

2. Users & Success Criteria

Operator (Tyler): technically deep, lives in CLI/Emacs, runs the homelab. Performs setup, weekly review, and maintenance. **Viewer (spouse):** non-technical; interacts only with the Paisa dashboard. Her single question: “how much is left in each category?”

- New bank transactions appear in the journal every morning with no manual steps and zero duplicates.
- The dashboard never overstates remaining budget: pending (AI-categorized) transactions draw down envelopes immediately.
- Weekly review is vendor-grouped and takes under ten minutes; every vendor correction becomes a permanent rule.
- All data is plain text in a git repository; ``hledger check`` passes at every commit.
- Any component (including every script built here) can be replaced without touching the data.

3. Scope

In scope (build these)

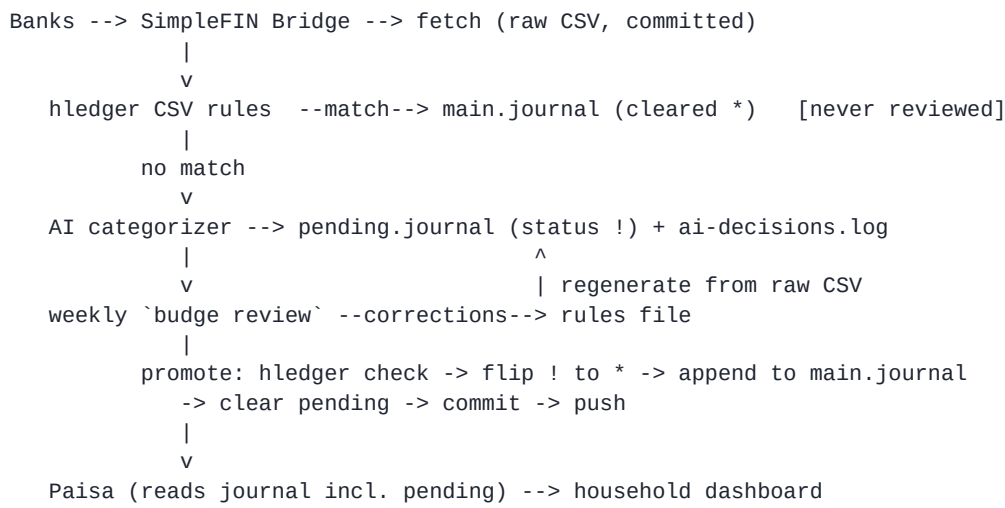
- **Setup script** — interactive bootstrap: prereqs, secrets, SimpleFIN claim, repo scaffold, 90-day backfill with computed opening balances; finishes by invoking the budget wizard.
- **Budget planning wizard** (``budge plan``) — standalone tool, bootstrap + re-assessment modes (\$7.8): analyzes real spending to propose the chart of accounts, envelope amounts against income and goals, and 20–30 starter merchant rules.

- **Fetch/import pipeline** — daily SimpleFIN pull → raw CSV → rules-first import → AI suggestions for the remainder → commit.
- **Categorizer** — provider-configurable AI client (Ollama cloud initially; OpenAI/Anthropic selectable) writing pending journal entries plus an audit log.
- **Review/promote CLI** — vendor-grouped review, rule-writing corrections, pending regeneration, validated promote-and-push.
- **agent.md** — the categorizer's instruction template (categories, conventions, transfer policy, output contract).
- **systemd units & timers** — fetch/import, AI categorization, git push, with OnFailure notification to OpenClaw.
- **Paisa deployment config** — container/service definition pointed at the journal (stock Paisa, no modifications).

Out of scope (do NOT build)

- Any modification to hledger, Paisa, or SimpleFIN Bridge themselves.
- Dashboard authentication or remote access (handled separately by the operator).
- Rental-property accounting structure (rent deposits are simply income:rent in the household books).
- Any path by which the AI or OpenClaw writes to the journal, the rules file, or git without human action (see §8).
- Mobile apps, custom dashboards, or UI beyond the review CLI (Paisa is the UI).

4. Architecture & Data Flow



Key invariant: a transaction's category is decided at import but only trusted at promote. Pending entries are included in all reports (via an `include` of pending.journal and the `!` status flag), so budgets are drawn down immediately at the AI's best guess. Review operates on vendors, not transactions: a correction edits the rules file and **regenerates** pending.journal from the raw CSVs — pending is a derived artifact; raw CSVs and main.journal are the records.

5. Repository Layout

```
budge/
  main.journal          # source of truth; includes the files below
  pending.journal       # DERIVED: AI-categorized, status !
  budget.journal        # periodic transaction (~ monthly envelopes; written by wizard)
  accounts.journal      # account declarations (chart of accounts)
  household.md          # income, savings target, goals, decisions (§7.8)
import/
  rules/<account>.rules  # hledger CSV rules, one per account
  raw/YYYY-MM/*.csv     # immutable SimpleFIN output, committed
  state/                # hledger .latest dedup state files
ai/
  agent.md              # categorizer instructions (§7.3)
  ai-decisions.log       # append-only audit log (§7.6)
scripts/                # all glue code built by this project
systemd/                # unit + timer files (installed by setup)
.gitignore              # excludes secrets-shaped files (defense in depth)

~/config/budge/secrets.env # OUTSIDE the repo, chmod 600:
SIMPLEFIN_ACCESS_URL, AI_API_KEY, etc.
```

6. Configuration

Key	Values / notes
ai.provider	openai-compatible anthropic. Ollama (cloud or local) and OpenAI share the openai-compatible path; switching to a local model is a base_url change only.
ai.base_url / ai.model	Endpoint and model name. Initial: Ollama cloud.

Key	Values / notes
ai.api_key	Read from secrets.env, never from the repo.
simplefin.access_url	Stored by setup after the one-time claim exchange (§7.1). The setup token itself is never persisted.
schedule.fetch / schedule.categorize / schedule.push	systemd OnCalendar expressions; defaults daily ~06:00 local, categorize after fetch, push after both.
notify.openclaw	Endpoint/mechanism for failure events (§7.7).

The AI request payload is limited to: payee/description, amount, date, and source account name — plus the category list and recent examples from agent.md. Account numbers, balances, and full history are never sent. This is a data-minimization contract, enforced in code regardless of provider.

7. Functional Requirements

7.1 Setup script (interactive, idempotent)

- Installs prerequisites (hledger, git, Python deps, Paisa container) on Debian 13; safe to re-run.
- Collects inputs: SimpleFIN setup token, AI provider/key, schedules, GitHub remote/auth.
- **Performs the SimpleFIN claim exchange:** exchanges the one-time setup token for the permanent access URL and stores only the access URL in secrets.env. Must clearly error if given an already-claimed token.
- Scaffolds the repo per §5 and writes .gitignore. Setup seeds only the transfer/payment rule patterns (§7.3); merchant rules are produced by the budget wizard from real data.
- **90-day backfill:** fetches available history, then computes and writes opening balance entries dated the day before the window, posted against equity:opening-balances, such that opening + transactions = the current balance SimpleFIN reports. Appends a balance assertion at today's date per account.
- If the assertion fails, do not abort silently: print the discrepancy, the arithmetic, and the likely cause (typically a bank-pending transaction included in balance but absent from the transaction list), and offer to proceed with an adjusted assertion or halt.
- **Invokes the budget planning wizard (§7.8) in bootstrap mode** after backfill completes, so the wizard analyzes real transactions. Setup must succeed even if the wizard is skipped (budgets can be planned later by running `budge plan`).
- Installs and enables the systemd units/timers; prints a final “what happens next” summary.

7.2 Fetch & import pipeline (daily timer)

- Pulls new transactions per account from the SimpleFIN access URL; writes immutable raw CSVs under import/raw/ and commits them.
- Runs `hledger import` with the per-account rules files; rule-matched transactions enter main.journal with cleared status (*). Dedup via hledger's .latest state plus SimpleFIN transaction IDs — a transaction must never import twice, including across re-runs and backfills.
- Unmatched transactions are handed to the categorizer (7.4).
- Appends a balance assertion using SimpleFIN's reported balance; an assertion failure is a pipeline failure (alerts fire, §7.7) — continuous reconciliation.
- Final step: single git commit capturing raw CSV + journal changes atomically. Push is a separate timer.

7.3 Transfer & credit-card handling (clearing account)

Card payments and inter-account transfers appear in *both* account feeds and must never be counted as spending (single- or double-counted). Mechanism:

- All transfer-shaped transactions post against **assets:transfers** (clearing account): checking's feed records checking → transfers; the card's feed records transfers → liabilities:card.
- Rules files must catch transfer/payment patterns (e.g. “PAYMENT THANK YOU”, “ONLINE TRANSFER”, ACH descriptors) **before** the AI ever sees them; setup seeds these patterns per account.
- agent.md instructs the model: anything transfer/payment-shaped is returned flagged as transfer, never assigned a spending category.
- assets:transfers must net to zero once both sides post; a persistent nonzero balance is surfaced in review as a warning (a missing or miscoded side).

7.4 AI categorizer (timer, after fetch)

- Provider-configurable client per §6; prompt assembled from agent.md (category list from accounts.journal, household conventions, transfer policy, output contract, recent approved examples).
- Output contract: structured (JSON) per transaction — category, confidence (high/medium/low), one-line rationale, transfer flag. Malformed model output is rejected and the transaction stays uncategorized in pending (expenses:uncategorized, status !) rather than guessed at.
- Writes pending.journal entries with status `!`, the AI's category, and a comment carrying confidence + rationale.
- Appends every decision to ai/ai-decisions.log (timestamp, txn id, payee, amount, suggestion, confidence, model, accepted/overridden — the last field updated at promote). Append-only; never rewritten.
- The categorizer never touches main.journal, the rules files, or git history. Its entire write surface is pending.journal and the log.

7.5 Review & promote CLI (`budge review`)

- Presents pending grouped by payee: count, total, suggested category, lowest confidence in group. Must handle the day-one case (hundreds of backfilled transactions) gracefully — sorted by count desc so big vendors are dispatched first.
- Per group, three actions: **approve**; **correct vendor** — writes/updates a rule in the rules file, then regenerates pending.journal from raw CSVs so all matching pending entries update at once; **correct single transaction** — one-off edit, no rule.
- Escape hatch: `budge review --edit` opens pending.journal in \$EDITOR.
- **Promote** (explicit final step): run `hledger check` (hard gate — any failure stops everything); flip ! to *; append entries to main.journal; truncate pending.journal; update ai-decisions.log accepted/overridden fields; one commit containing journal + rules changes together; push.
- Rule edits made during review are validated by the same `hledger check` gate before commit (catches shadowing/malformed rules).

7.6 Logging

Beyond ai-decisions.log: all timers log to journald with consistent unit naming (budge-fetch, budge-categorize, budge-push). No custom log infrastructure.

7.7 Failure notification (OpenClaw)

- Each unit has OnFailure= wired to a notifier that sends OpenClaw: unit name, exit status, last ~20 lines of journalctl.
- Weekly nudge: “review ready — N transactions pending” after the last categorize run before the operator's review day.
- **Policy (hard boundary)**: OpenClaw's role is read-only triage and notification. It must not edit the journal, rules, or secrets, must not retry destructive operations, and must not promote. Nothing in this project may grant it write access to the repo.

7.8 Budget planning wizard (`budge plan`)

A standalone interactive tool, invoked by setup at bootstrap and run by the operator thereafter (a quarterly timer may *nudge* via OpenClaw that re-assessment is due, but the wizard only ever runs when a human runs it). Core stance: **actuals are descriptive, budgets are prescriptive** — the wizard organizes the user's own

data against the user's own stated constraints and proposes; the human decides. It never writes any file without explicit confirmation.

Intake (exactly three prompts, stored in household.md)

- Monthly take-home income.
- Monthly savings target.
- Freeform text: "What are you saving for, and what's changing in the next year?" — passed to the model as planning context (anticipates what history cannot show, e.g. an arriving child).
- household.md is git-versioned and editable directly; the wizard reads it on every run and appends each run's decisions (proposed vs. chosen), accumulating the "why" behind budget history. No demographic forms — household composition is already encoded in the spending data.

Bootstrap mode (fresh repo, after backfill)

- AI pass over the raw backfilled transactions clusters merchants into proposed categories. One analysis yields three confirmable artifacts: (a) the chart of accounts → accounts.journal; (b) per-category monthly amounts → a `~ monthly` periodic transaction in budget.journal; (c) the merchant→category mapping → 20–30 starter rules in the rules files (directly shrinking the day-one review pile).
- Amount proposal is constrained arithmetic: income – savings target = spending ceiling; envelopes must sum within it. Observed monthly actuals are shown per category as the default; the human types the target. If targets exceed the ceiling, the wizard surfaces the gap and the categories with the most discretionary slack — it does not resolve the tension itself.
- Must disclose the 90-day sample bias: annual/seasonal expenses (insurance, registration, holidays) are invisible; propose a sinking-fund envelope for irregulars rather than extrapolating the quarter as the year.
- Each artifact is presented as a reviewable proposal (editable before write); confirmed artifacts are written and committed with `hledger check` passing.

Re-assessment mode (existing books) — minimal in v1

- Runs `hledger balance --budget` over the trailing N months (default 3); reports variance per category with streaks ("dining 32% over for 3 consecutive months") and dormancy ("no gym transactions in 90 days").
- Reads household.md context against actuals ("childcare materialized at \$1,100/mo vs. nothing budgeted; savings target no longer arithmetic-compatible — revisit?").
- Output: a proposed diff to budget.journal (occasionally accounts/rules suggestions, e.g. splitting a category), shown for approval, then committed. Never auto-applied.
- Richer guidance (goal decomposition, tradeoff exploration) is explicitly v2; v1 builds the data foundation (household.md + decision history) those versions will use.

Hard boundaries

- No external benchmarks or norm claims ("families your size typically spend..."); widely-known arithmetic framings of the user's own numbers (e.g. 50/30/20 as a presentation lens) are acceptable.
- Surfaces arithmetic conflicts between goals; never lectures or auto-prioritizes between them unprompted. The wizard is a bookkeeping assistant, not a financial advisor.

8. AI & Agent Policy (applies to every component)

- The AI suggests; humans decide. No AI-assigned category becomes cleared (*) without the operator running promote.
- Every AI decision is logged before it is used. The log is the evidence base for any future loosening (e.g. auto-promoting high-confidence suggestions) — such loosening is explicitly a v2 decision, not to be built now.
- Deterministic rules always run before the AI; the system's trajectory is rules absorbing vendors so the AI's share shrinks.
- Data minimization per §6 regardless of provider.

9. Acceptance Criteria & Test Cases

#	Scenario	Pass condition
A1	Fresh setup on clean Debian 13 LXC	Setup completes; `hledger check` passes; Paisa serves the dashboard; timers active.
A2	90-day backfill	Opening balances + imported txns reconcile to SimpleFIN's reported balance via passing assertions; equity:opening-balances holds the remainder.
A3	Duplicate prevention	Re-running fetch/import N times produces zero duplicate transactions.
A4	Card payment, both accounts connected	Payment posts through assets:transfers from both feeds; clearing account nets to zero; no expense recorded.
A5	Transfer reaches the AI anyway	Model returns transfer flag per agent.md; entry posts to assets:transfers, surfaced in review as a rules gap.
A6	Pending visibility	An unreviewed AI-categorized txn reduces its envelope's remaining amount in Paisa immediately.
A7	Vendor correction	Correcting one vendor in review updates every matching pending txn (regeneration), writes the rule, and the vendor never re-enters pending.
A8	Promote gate	A deliberately malformed pending entry causes `hledger check` to halt promote with nothing written, committed, or pushed.
A9	Malformed AI output	Categorizer rejects it; txn lands as expenses:uncategorized status !; logged.
A10	Pipeline failure	Killing the fetch mid-run fires OnFailure; OpenClaw receives unit, status, and log tail.
A11	Idempotent setup re-run	Second run changes nothing destructive; budget wizard detects existing budget.journal and skips.

#	Scenario	Pass condition
A12	Dry run	Every script supports --dry-run printing intended actions with zero writes.
A13	Wizard bootstrap	From backfilled data + the three intake prompts, the wizard proposes chart of accounts, envelope amounts summing within income – savings, and starter rules; nothing is written until each artifact is confirmed; result passes `hledger check`.
A14	Over-ceiling targets	Targets exceeding the spending ceiling produce a surfaced gap and slack candidates — the wizard does not silently rescale or pick cuts.
A15	Re-assessment	With ≥ 3 months of categorized data, the wizard produces a variance report and a proposed budget.journal diff; declining the diff leaves the repo untouched; household.md records the run's decisions.

10. Risks & Mitigations

- **Bank drops SimpleFIN support** → manual CSV export feeds the identical rules/import path.
- **Day-one review overload** (backfill) → vendor grouping sorted by count; expected one ~hour session that seeds 30–40 rules.
- **Assertion failures from bank-pending txns** → explained-discrepancy flow in setup (§7.1) and tolerant alerting language in daily runs.
- **AI provider outage** → unmatched txns accumulate as uncategorized-pending; books remain correct in total; categorization catches up next run.
- **Rules file drift/shadowing** → `hledger check` gate on every rule edit and every promote.

11. Open Items (inputs owed by the operator)

- Wizard intake answers at setup time: monthly take-home, savings target, goals/upcoming-changes paragraph (the category list itself is now drafted by the wizard from real data — the operators react to the proposal rather than authoring it).
- GitHub remote URL and auth method for the journal repo.
- OpenClaw notification endpoint details.

Build order suggestion: 7.1 setup + scaffold → 7.2 fetch/import → 7.3 transfer rules → 7.8 wizard (bootstrap mode) → 7.4 categorizer → 7.5 review/promote → 7.7 notifications → 7.8 re-assessment mode → acceptance pass A1–A15.